dida-trade: State-Machine Replication with Multi-Paxos, Fast Paxos and Vertical Paxos

Diogo Noronha da Cruz, n.º 106591 João Pedro Correia e Sousa, n.º 99991
Tomás Dias Monteiro, n.º 106211
Instituto Superior Técnico, Universidade de Lisboa
Design and Implementation of Distributed Applications 2026/27 — Group 38

1. Steps implemented

We implemented Steps 1–5 (Ultimate goal): the real phase 1 and the debug modes, Multi-Paxos, Fast Paxos, a causally ordered `balance`, and dynamic membership with state transfer. Progress comes only from protocol events, plus manual reconfiguration from the console.

2. Multi-Paxos (Steps 1–2)

A leader runs one phase 1 per ballot, starting at its first unexecuted position. Acceptors reply with every vote they cast from that position on, and flag the values they know are decided. For each position the leader re-proposes the decided value, or else the value voted in the highest ballot. Gaps are filled with no-ops. Each pending command then gets its own proposer thread, which reserves the next position under a lock and sends phase 2 without waiting for earlier positions to be decided. Acceptors send their vote directly to every learner, and positions are executed strictly in order.

Auxiliary structures. `RequestHistory` keeps client requests in two maps, *pending* and *processed*, keyed by request id. A request is added to *pending* when a client sends it, and moved to *processed* when its position is executed. If the same request id is decided in a second position, it is found in *processed* and skipped. The proposer keeps `proposed_requests`, the set of request ids it has already sent to the acceptors. An id is added when a proposer thread is started for it and removed when it executes. The set is cleared on a ballot change, so that the new leader proposes again whatever its phase 1 did not recover. Each log entry (`PaxosInstance`) keeps the acceptor's own vote separately from the learner's tally, a map from value to the *set of acceptors* that voted it. This matters in fast ballots, where acceptors may vote for different values in the same position.

3. Fast Paxos (Step 3)

The leader of a fast ballot runs phase 1 and then sends an *any* message, which opens every position from the first free one. Since clients send each request to every replica, every replica acts as a fast proposer once it has received the *any*.

The acceptor chooses the position. A fast proposal carries no position, because without a leader there is no one to serialize reservations. Each acceptor places the request in its lowest open position. A request it has already voted for in this ballot keeps its position, and a request it already knows is decided gets none. Otherwise, a late duplicate would occupy the next position on that acceptor alone and collide with the following command. A proposal that reaches an acceptor before the *any* waits for it, instead of being refused.

Quorums. We extended the `Schedule` interface with `fastquorum` (the super-majority) and `recoveryquorum` (the promises needed in phase 1), alongside the existing `quorum`. With 3 acceptors the fastquorum is 3 and the recoveryquorum is 2. With 5 acceptors they are 4 and 3.

Collisions and recovery. A learner declares a collision when a fast quorum of distinct acceptors has voted in a position without agreeing on a value. It then moves to the next ballot, whose scheduled leader starts phase 1 on its own; this is the automatic reconfiguration of the specification. For each position, the new leader looks at the highest ballot voted by its recovery quorum. If that ballot was fast, a value may have been chosen only if that value was the majority in any quorum of acceptors. Such a value must be re-proposed; otherwise any reported value is safe.

Auxiliary structures. Each proposer keeps `fast_marks`, a map from position to the request it saw voted there in the current fast ballot. An entry is added when an acceptor reply or a local vote places a request, and removed when that position is decided. If a different request won the position, the losing request is also removed from `proposed_requests`, so that it is proposed again.

4. Causal `balance` (Step 4)

All writes go through one totally ordered log, so a client's causal past fits in a single integer: the number of log positions it has observed. Each write reply carries the number of positions the replica had executed. The client keeps the maximum, `seen_slot`, and sends it with `balance` as `min_slot`. A replica answers `balance` locally, without consensus, once it has executed at least `min_slot` positions; until then the request waits. The reply carries the replica's position count, which the client folds into `seen_slot`. The state machine and the position counter are updated under the same lock that `balance` reads them with.

5. Dynamic membership and state transfer (Step 5)

Completed ballot. A leader's phase 1 queries the acceptors of every ballot from the last *complete* ballot c up to its own, and needs a recovery quorum of each set. A ballot b becomes complete only when three conditions hold: its leader finished phase 1; every position phase 1 recovered (decided ones included) is accepted by a quorum of b's acceptors; and, if the acceptor set changed, a quorum of the new acceptors has received the decided log (`installstate`). A `CompletionTracker` per ballot counts the outstanding recovered positions and the transfer, and only sets c when both are done. The leader returns c to the console, which passes it to the next leader with `ballot` or `activate`. A replica that never learns c keeps a lower value, which only makes its phase 1 ask more acceptors.

Pull transfer. Execution blocks at a position that is not decided while later ones are, or at a position that is decided but whose client request never reached this replica. The replica then asks every peer (`fetchstate`) for the decided positions from there on, *with the commands*. `catch_up_requested_instance` records the position it last asked about, so it asks at most once per blocked position. `max_decided` holds the highest decided position seen, which is what reveals a gap. We transfer the log rather than a snapshot. Replay is deterministic and also rebuilds `RequestHistory`, which a client request arriving after a transfer needs in order to find its stored reply.

Recovery. A restarted replica has lost its promises and votes. Before acting as an acceptor, it fetches the log from a recovery quorum of peers and replays it. If the system was already running, it also abstains from every ballot up to the highest ballot its peers report (`abstain_upto`). It may have voted in those ballots before crashing, and voting again could contradict an earlier vote. Abstaining is answered as "unavailable", so the other acceptors can still form a quorum. A peer that receives `fetchstate` resets its gRPC reconnect back-off towards the sender, which could otherwise reject calls to the restarted replica for up to two minutes.

6. Limitations

Without timeouts, a fast ballot that stalls with an acceptor down, or a ballot whose leader has crashed, is unblocked only by a collision or by a `ballot` command from the console. Without stable storage, recovery relies on abstention. This assumes that some peer has seen every ballot the crashed replica took part in, which holds because every ballot needs a quorum, but it is not a proof. The transferred log grows without bound.

Use of AI

"All design choices and the code have been by us. The following tools: Copilot, Claude; have been used for this purpose: implementing code, sketching the report."

Authorship

We, Diogo Noronha da Cruz, n.º 106591, MEIC; João Pedro Correia e Sousa, n.º 99991, MEIC; Tomás Dias Monteiro, n.º 106211, MEIC, hereby declare, on our honor, that the work entitled "dida-trade", carried out as part of the Design and Implementation of Distributed Applications course, resulted from the active participation of the following group members. The individual contributions were: Diogo Cruz — debug modes and phase 1 (Step 1), Multi-Paxos (Step 2), Fast Paxos (Step 3), dynamic membership and state transfer (Step 5), automated tests, report; João Sousa — development environment, debug modes and phase 1 (Step 1), Multi-Paxos (Step 2), causally ordered `balance` (Step 4), automated tests; Tomás Monteiro — Multi-Paxos (Step 2), Fast Paxos (Step 3), dynamic membership and Vertical Paxos (Step 5), automated tests, report, overall review and fixes. We declare that this work is original and complies with the applicable academic regulations.

Lisboa, October 16th 2026 Signatures: